

Лабораторная работа №3

Простейшая нейронная сеть с одним скрытым слоем: реализация на Python+NumPy и альтернативно на Keras

Лабораторная работа №3 направлена на освоение базового цикла создания модели машинного обучения в контексте ИС: подготовка среды, подготовка данных, реализация и обучение простой нейросети (полносвязный MLP с одним скрытым слоем), анализ качества и оформление отчёта. Основной трек — «чистая» реализация на **Python+NumPy** (чтобы понять математику и механику обучения), альтернативный трек — реализация на **Keras (TensorFlow)** (чтобы закрепить индустриальный подход и типовой пайплайн обучения/оценки). Опорные элементы: прямой проход, функция потерь **MSE** (mean squared error) и обратное распространение ошибки (backpropagation) с градиентным спуском. Формула MSE в документации Keras записывается как среднее квадратов разности истинных и предсказанных значений. [1]

Цели, результаты обучения и связь с компетенциями бакалавра ИС

Лабораторная работа соотносится с общепрофессиональными компетенциями направления 09.03.02 «Информационные системы и технологии». Например, ФГОС фиксирует, что выпускник должен быть способен разрабатывать алгоритмы и программы для практического применения в ИС (ОПК-6), выбирать платформы и инструментальные средства (ОПК-7), применять математические модели и методы проектирования (ОПК-8), а также устанавливать ПО для ИС (ОПК-5). [5]

Цели (для преподавателя и студента):

- Понять «внутренности» обучения нейросети: как считаются активации, ошибка и градиенты; почему веса обновляются в направлении уменьшения ошибки. [6]
- Научиться собирать воспроизводимую среду разработки (venv/conda), фиксировать зависимости и запускать эксперименты без «конфликтов библиотек». [7]
- Реализовать MLP (1 скрытый слой) на NumPy и сравнить с Keras-реализацией по удобству, скорости разработки и качеству результата (графики loss/accuracy). [8]

Ожидаемые результаты обучения (learning outcomes):

- Студент объясняет, что такое прямой проход, функция потерь MSE и backpropagation, и может вручную разложить вычисления на шаги и размерности тензоров. [9]
- Студент пишет и запускает обучающий цикл на NumPy (forward → loss → backward → update), строит график обучения и делает вывод о сходимости. [10]
- Студент реализует аналогичную модель в Keras через Sequential, обучает через fit() и интерпретирует history и метрики. [11]

Требования к среде и установка

В лабораторной допускаются два «профиля» окружения: минимальный (NumPy) и расширенный (Keras/TensorFlow). Для TensorFlow актуальны ограничения по версиям Python;

официальная инструкция по установке через pip указывает поддерживаемые версии Python (например, 3.9–3.12) и требования к pip. [14]

Требования (рекомендуемо):

- Python: **3.11** (компромисс «совместимость/стабильность»); для TensorFlow сверяйтесь с поддержкой версий. [14]
- Библиотеки (минимум): numpy, matplotlib. [15]
- Библиотеки (варианты):
- Вариант А (классический ML-пайплайн): scikit-learn (датасеты, split, scaler). [16]
- Вариант В (DL-фреймворк): tensorflow (включая Keras API), либо отдельно Keras-пакеты в зависимости от выбранной схемы. [17]

Установка через venv + pip (рекомендуется в учебных аудиториях)

Python официально описывает виртуальные окружения и активацию (Windows / Unix-подобные). [18]

Python Packaging User Guide даёт пошаговую схему установки пакетов в venv и работы с requirements. [19]

1) Создать окружение в папке проекта:

```
python -m venv .venv
```

2) Активировать:

- Windows (PowerShell):

```
.\.venv\Scripts\activate
```

- Linux/macOS:

```
source .venv/bin/activate
```

[20]

3) Обновить pip и поставить зависимости:

```
python -m pip install --upgrade pip
pip install numpy matplotlib scikit-learn
pip install tensorflow
```

Замечание: если TensorFlow не требуется (делаете только NumPy-вариант), последнюю строку опустите. Совместимость TensorFlow по Python смотрите в официальной инструкции. [14]

4) Зафиксировать зависимости для воспроизводимости:

```
pip freeze > requirements.txt
```

(Подход с requirements файлами — стандартная практика в Python-упаковке.) [19]

Установка через conda (вариант для сложных зависимостей)

Conda официально описывает управление Python-версией в окружении и активацию. [21]
Anaconda подчёркивает, что `pip install` не стоит выполнять в `base` и что `pip`-операции лучше делать в отдельном окружении и «в конце сборки окружения». [22]

```
conda create -n is_lab3 python=3.11
conda activate is_lab3
conda install numpy matplotlib scikit-learn
pip install tensorflow
```

Если используете `pip` внутри `conda` — соблюдайте рекомендацию «`pip` в конце», чтобы уменьшить риск конфликтов. [22]

Практическая часть: нейросеть с одним скрытым слоем

В этой лабораторной нейросеть — это функция с параметрами (весами и смещениями), которую мы подбираем так, чтобы минимизировать ошибку на данных. В учебном варианте используем полносвязную сеть (MLP): вход → скрытый слой → выход.

Математическая база

Прямой проход (forward pass):

- Считаем линейную комбинацию: $z = XW + b$, затем нелинейность $a = \sigma(z)$ (например, сигмоида).
- Для двухслойной сети:
- $z_1 = XW_1 + b_1$, $a_1 = \sigma(z_1)$
- $z_2 = a_1W_2 + b_2$, $\hat{y} = \sigma(z_2)$ (для бинарного выхода)

Функция потерь MSE:

- В Keras документации: `loss = mean(square(y_true - y_pred))`. [23]

Backpropagation (обратное распространение):

Идея: найти производные ошибки по параметрам и сделать шаг градиентного спуска. В лекционных материалах по `backprop` (для случая среднеквадратичной ошибки) приводятся формулы δ -ошибок по слоям и общий алгоритм: прямой проход → вычисление δ на выходе → распространение δ назад → градиенты по W и b . [6]

Mermaid-диаграммы (вставьте в отчёт и, при желании, на слайды/раздатку)

Структура сети (1 скрытый слой):

flowchart LR

```
X[Вход X (n_features)] -->|W1,b1| Z1[z1 = XW1 + b1]
Z1 --> A1[a1 = σ(z1)]
A1 -->|W2,b2| Z2[z2 = a1W2 + b2]
Z2 --> Yhat[ŷ = σ(z2)]
Yhat --> L[Loss = MSE(y, ŷ)]
```

Алгоритм обучения:

flowchart TD

```
A[Инициализация W,b] --> B[Для каждой эпохи]
B --> C[Forward: получить  $\hat{y}$ ]
C --> D[Loss: MSE(y,  $\hat{y}$ )]
D --> E[Backward: вычислить градиенты]
E --> F[Update:  $W, b = W, b - lr * grad$ ]
F --> G{Критерий остановки?}
G -->|нет| B
G -->|да| H[Сохранить метрики/графики/модель]
```

Данные для тестирования (генерируемые и реальные)

Синтетика 1: XOR (бинарная классификация)

- X — 4 точки (0/1), y — 0 или 1.
- Преимущество: легко проверить «на глаз», сеть с одним скрытым слоем решает XOR, а линейная модель — нет.

Синтетика 2: регрессия $\sin(x)$

- X — равномерная сетка, $y = \sin(x) + \text{шум}$.
- Преимущество: хорошо видно поведение MSE и сходимость.

Реальные учебные данные: Iris (scikit-learn)

- Iris — классический небольшой датасет; документация описывает размеры и признаки. [4]
- Для упрощения берём две категории (бинарная задача) и используем train/test split. [13]
- Для стабильного обучения стандартизируем признаки (StandardScaler). [12]

Реализация 1: «чистый» NumPy (обязательный трек)

Ниже — шаблон, который студент должен воспроизвести, комментируя размерности. Старайтесь использовать матричные операции (@) вместо циклов по объектам, поскольку матричное умножение — стандартный «язык» для нейросетей. [3]

Шаг 1. Заготовки и воспроизводимость

NumPy рекомендует использовать default_rng как «конструктор генератора» для случайных чисел. [24]

```
import numpy as np
import matplotlib.pyplot as plt
```

```
rng = np.random.default_rng(42) # фиксируем seed для воспроизводимости
```

Шаг 2. Функции активации и их производные

```
def sigmoid(x):
```

```
    # защита от переполнения: при больших |x| exp может "взорваться"
    x = np.clip(x, -50, 50)
    return 1 / (1 + np.exp(-x))
```

```
def sigmoid_derivative(sigmoid_x):
```

```
# если уже посчитали  $s = \text{sigmoid}(z)$ , то  $s' = s*(1-s)$   
return sigmoid_x * (1 - sigmoid_x)
```

Шаг 3. MSE и accuracy (для бинарной задачи)

Формулу MSE можно фиксировать как среднее квадратов ошибок. [23]

```
def mse(y_true, y_pred):  
    return np.mean((y_true - y_pred) ** 2)
```

```
def accuracy_binary(y_true, y_pred):  
    y_hat = (y_pred >= 0.5).astype(int)  
    return np.mean(y_hat == y_true)
```

Шаг 4. Данные XOR (минимальный тест)

```
X = np.array([  
    [0., 0.],  
    [0., 1.],  
    [1., 0.],  
    [1., 1.]  
)
```

```
y = np.array([[0.], [1.], [1.], [0.]]) # XOR
```

Шаг 5. Инициализация весов и размерности

```
n_samples, n_features = X.shape  
n_hidden = 4  
n_outputs = 1
```

```
W1 = rng.normal(0, 1, size=(n_features, n_hidden)) * 0.5  
b1 = np.zeros((1, n_hidden))
```

```
W2 = rng.normal(0, 1, size=(n_hidden, n_outputs)) * 0.5  
b2 = np.zeros((1, n_outputs))
```

Шаг 6. Цикл обучения (forward → loss → backward → update)

В backprop-конспектах для MSE приводится общий алгоритм: прямой проход, затем δ на выходе, затем распространение δ назад и вычисление градиентов по W/b . [6]

```
lr = 0.5  
epochs = 2000
```

```
loss_history = []  
acc_history = []
```

```
for epoch in range(epochs):  
    # --- Forward ---  
    z1 = X @ W1 + b1  
    a1 = sigmoid(z1)
```

```

z2 = a1 @ W2 + b2
y_pred = sigmoid(z2)

# --- Metrics ---
loss = mse(y, y_pred)
acc = accuracy_binary(y, y_pred)

loss_history.append(loss)
acc_history.append(acc)

# --- Backward ---
# dLoss/dy_pred для MSE: 2*(y_pred - y)/N (коэффициент 2 можно поглотить lr)
N = n_samples
dL_dy = (y_pred - y) / N

dy_dz2 = sigmoid_derivative(y_pred)
delta2 = dL_dy * dy_dz2 # (N,1)

dW2 = a1.T @ delta2 # (hidden,1)
db2 = np.sum(delta2, axis=0, keepdims=True)

delta1 = (delta2 @ W2.T) * sigmoid_derivative(a1) # (N,hidden)
dW1 = X.T @ delta1
db1 = np.sum(delta1, axis=0, keepdims=True)

# --- Update ---
W2 -= lr * dW2
b2 -= lr * db2
W1 -= lr * dW1
b1 -= lr * db1

# --- Логирование ---
if epoch % 200 == 0:
    print(f"epoch={epoch:4d} loss={loss:.4f} acc={acc:.3f}")

```

Ожидаемый вывод (примерный):

- В начале loss около 0.25–0.30, accuracy около 0.5.
 - К концу обучения accuracy часто приближается к 1.0, loss падает.
- Точные числа зависят от инициализации и lr, поэтому важно фиксировать seed. [24]

Шаг 7. Визуализация графиков (loss/accuracy)

Matplotlib описывает базовую работу с pyplot как MATLAB-подобный интерфейс построения графиков. [25]

```

plt.figure()
plt.plot(loss_history)
plt.title("Training loss (MSE)")
plt.xlabel("Epoch")
plt.ylabel("Loss")

```

```
plt.grid(True)
plt.savefig("loss_curve.png", dpi=150)
```

```
plt.figure()
plt.plot(acc_history)
plt.title("Training accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.grid(True)
plt.savefig("acc_curve.png", dpi=150)
plt.show()
```

Типовые ошибки в NumPy-реализации и отладка

- **Несовпадение размерностей (ValueError: shapes ... not aligned)**: проверяйте формы матриц перед @. Часто путают (N,features) vs (features,N). Рекомендация: печатать X.shape, W1.shape, a1.shape. [3]
- **Плохая сходимость / скачет loss**: слишком большой lr или плохая инициализация; уменьшите lr в 2–10 раз и проверьте, падает ли loss монотоннее.
- **NaN в loss**: переполнение экспоненты в sigmoid; используйте clip (как в шаблоне) и/или уменьшите lr.
- **Accuracy не растёт**: ошибка в производной сигмоиды (перепутали $\sigma(z)$ и z), или неправильно определили точность (например, сравниваете float с int без порога 0.5).

Реализация 2: Keras (альтернативный трек)

Keras-подход показывает «как делают в индустрии»: модель собирается как стек слоёв (Sequential), обучение идёт через fit(), а история метрик доступна через History. [26]

Шаг 1. Установка TensorFlow (если не установлен)

Официальная инструкция по pip-установке TensorFlow перечисляет требования (Python-версии, pip и т.д.). [14]

```
pip install tensorflow
```

Шаг 2. Код модели (1 скрытый слой) и обучение на XOR

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
```

```
X = np.array([[0.,0.],[0.,1.],[1.,0.],[1.,1.]], dtype=np.float32)
y = np.array([[0.],[1.],[1.],[0.]], dtype=np.float32)
```

```
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(2,)),
    tf.keras.layers.Dense(4, activation="sigmoid"),
    tf.keras.layers.Dense(1, activation="sigmoid"),
])
```

```
model.compile(
    optimizer="adam",
    loss=tf.keras.losses.MeanSquaredError(), # MSE согласно определению
    metrics=["accuracy"]
)
```

```
history = model.fit(X, y, epochs=500, verbose=0)
```

```
print("Final:", history.history["loss"][-1], history.history["accuracy"][-1])
```

- Sequential и его применимость к «стеку слоёв» описаны в документации. [27]
- Назначение loss-функций (что минимизируем) описано в Keras Losses. [28]
- MSE как $\text{mean}(\text{square}(y_{\text{true}} - y_{\text{pred}}))$ фиксируется в документации. [1]

Шаг 3. Графики обучения (loss/accuracy) из history

```
plt.figure()
plt.plot(history.history["loss"])
plt.title("Keras training loss (MSE)")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)
plt.savefig("keras_loss.png", dpi=150)
```

```
plt.figure()
plt.plot(history.history["accuracy"])
plt.title("Keras training accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.grid(True)
plt.savefig("keras_acc.png", dpi=150)
plt.show()
```

Смысл fit()/evaluate()/predict() и работа со встроенными циклами обучения описаны в гайде TensorFlow по обучению «встроенными методами». [29]

Таблицы, задания, оценивание, отчёт и контроль

Таблица план-тайминга выполнения (рекомендуемая структура занятия и самостоятельной части)

Этап	Содержание	Время (аудитория)	Самостоятельно
Подготовка окружения	venv/conda, установка пакетов, проверка импортов	15 мин	10 мин
NumPy-реализация	forward, MSE, backward, update	35 мин	30–60 мин
Тесты на данных	XOR + обсуждение, подбор lr/epochs	15 мин	20 мин

Этап	Содержание	Время (аудитория)	Самостоятельно
Визуализация	графики loss/accuracy в matplotlib	10 мин	10 мин
Keras-вариант	Sequential, compile, fit, history	15 мин	20–40 мин
Оформление отчёта	структура, выводы, скриншоты/графики	10 мин	60 мин

Критерии оценки и контрольные точки (пример, 100 баллов)

- **Среда и воспроизводимость (10):** есть requirements.txt/environment.yml, инструкции запуска, фиксированный seed. [30]
- **NumPy-модель (35):** корректная реализация forward/backward/update, комментарии по размерностям, отсутствие «магии». Backprop-логика соответствует общему алгоритму. [6]
- **Эксперименты и данные (15):** XOR + один дополнительный набор (sin или Iris-бинаризация), корректное разделение train/test при реальных данных. [31]
- **Метрики и графики (10):** построены loss/accuracy, краткая интерпретация («сходится/не сходится, почему»). [32]
- **Keras-вариант (15):** рабочая модель, обучение, сохранение графиков по history, краткое сравнение с NumPy. [11]
- **Отчёт и выводы (15):** структура, объяснение математики и ошибок, вывод о плюсах/минусах подходов, аккуратность.

Таблица требований к сдаче (структура файлов)

Элемент	Имя/папка	Что внутри	Обязательно
Код NumPy	src/nn_numpy.py	реализация сети + обучение + сохранение графиков	Да
Код Keras	src/nn_keras.py	модель Sequential + fit + графики	Да (или как доп. задание)
Зависимости	requirements.txt	версии библиотек	Да
Отчёт	report.pdf или report.md	цель, теория, код-фрагменты, графики, выводы	Да
Результаты	results/	loss_curve.png, acc_curve.png, keras_loss.png, keras_acc.png	Да
Данные	data/	(опционально) CSV, если использовали внешний набор	Нет

Таблица сравнения NumPy vs Keras

Критерий	NumPy-реализация	Keras-реализация
Понимание математики	Максимальное: явно пишем все формулы и градиенты [6]	Частично скрыто: фреймворк делает backprop сам
Скорость разработки	Медленнее	Быстрее: Sequential → compile → fit [33]
Риск ошибок	Высокий (shape, производные, lr)	Ниже (типовые ошибки меньше)

Критерий	NumPy-реализация	Keras-реализация
Масштабирование	Ограничено учебными задачами	Реальные проекты, большие датасеты
Диагностика	Нужно писать вручную	history, callbacks, инструменты экосистемы [29]

Задания

Обязательное (все выполняют):

- Реализовать MLP (1 скрытый слой) на NumPy для XOR, показать графики loss/accuracy и добиться accuracy ≥ 0.95 .
- Объяснить в отчёте: что такое W , b , z , a , $\hat{y}$, как считается MSE и как в общих чертах работает backprop. [34]

Дополнительные (для отличников):

- Сделать бинарную классификацию на Iris (выбрать 2 класса), разделить train/test и стандартизировать признаки. [16]
- Заменить MSE на cross-entropy и сравнить сходимость (короткий эксперимент).
- Реализовать mini-batch обучение и сравнить скорость/стабильность с full-batch.

Вопросы для самопроверки и тесты

MCQ (10):

1) Что делает venv?

- A) Создаёт отдельный интерпретатор и изолирует пакеты проекта.
- B) Удаляет системный Python.
- C) Компилирует NumPy в C.
- D) Ставит TensorFlow без pip.

2) Как в Keras определяется MSE?

- A) `mean(abs(y_true - y_pred))`
- B) `mean(square(y_true - y_pred))`
- C) `max(y_true - y_pred)`
- D) `sum(y_true)`

3) Почему нормализация/стандартизация может улучшить обучение?

- A) Скрывает ошибки измерений
- B) Делает признаки сопоставимыми по масштабу, что упрощает оптимизацию
- C) Удаляет выбросы автоматически
- D) Увеличивает размер датасета

4) Для чего нужен train/test split?

- A) Чтобы увеличить точность всегда
- B) Чтобы измерить качество на данных, не участвовавших в обучении
- C) Чтобы уменьшить память
- D) Чтобы не использовать seed

- 5) Что такое backpropagation?
- A) Метод построения графиков
 - B) Метод вычисления градиентов для обновления весов в нейросети
 - C) Алгоритм сортировки
 - D) Формат датасета
- 6) В формуле $z = XW + b$ что такое b ?
- A) Градиент
 - B) Смещение (bias), добавляется к линейной комбинации
 - C) Размер батча
 - D) Метрика
- 7) Что делает `model.fit()` в TensorFlow/Keras?
- A) Строит график loss
 - B) Запускает процесс обучения по данным (итеративно обновляет веса)
 - C) Только загружает датасет
 - D) Сохраняет модель в файл
- 8) В NumPy для матричного произведения предпочтительно использовать:
- A) `*`
 - B) `@` или `np.matmul`
 - C) `+`
 - D) `np.sum`
- 9) Что означает «короткое замыкание» в контексте оптимизации?
- A) Остановка обучения при NaN
 - B) Выполнение операций быстрее за счёт SIMD
 - C) Прерывание вычислений при достижении критерия остановки
 - D) Предотвращение переполнения `exp`
- 10) Какой источник случайности рекомендует NumPy для генерации?
- A) `np.random.RandomState()` всегда
 - B) `np.random.default_rng()`
 - C) только системный `random`
 - D) RNG не нужен

Короткие вопросы (5):

- 1) Запишите MSE в виде формулы.
- 2) Что такое «эпоха»?
- 3) Почему в sigmoid полезно ограничивать вход `clip`?

4) Назовите 3 типовые причины, почему loss не уменьшается.

5) Как в conda правильно сочетать conda и pip?

Рекомендации по оформлению отчёта (структура)

- **Титульный лист/шапка:** дисциплина, ЛР №3, ФИО, группа, дата, преподаватель.
- **Цель и постановка:** что строим (MLP 1 hidden layer), какие данные, какие метрики.
- **Теория:** forward, MSE, backprop (в 1–2 страницы), ссылка на формулу MSE и общий алгоритм backprop. [34]
- **Реализация:** фрагменты ключевого кода, пояснение размерностей, описание эксперимента (lr/epochs/seed).
- **Результаты:** графики loss/accuracy, таблица финальных метрик.
- **Обсуждение ошибок и отладки:** 2–3 проблемы, как нашли/исправили.
- **Вывод:** что получилось, что улучшили бы (например, другой loss, регуляризация).

Безопасность и этика при работе с данными

Даже учебные проекты должны формировать «правильные привычки». Общие принципы обработки персональных данных в GDPR включают законность, справедливость и прозрачность обработки. [38]

В российском правовом поле закон 152-ФЗ также закрепляет принципы законности и ограничения целей обработки персональных данных. [39]

Практические правила для ЛР:

- Используйте **открытые учебные датасеты** (Iris) или синтетику; не используйте реальные персональные данные студентов/клиентов. [4]
- Если берёте «реальные» данные из организации — требуйте обезличивание, минимизацию и ограничение доступа. [40]
- В отчёте фиксируйте источник данных и лицензионный статус (ссылка, условия использования).
- Для этического контекста ИИ: ориентируйтесь на принципы доверенного ИИ (прозрачность, справедливость, устойчивость, ответственность). [41]

Рекомендуемые источники (в приоритете официальные)

- Python: виртуальные окружения venv и активация. [35]
- Python Packaging: установка пакетов с pip и venv. [19]
- NumPy: broadcasting и матричное умножение (matmul, @). [42]
- NumPy RNG: default_rng и воспроизводимость. [24]
- Matplotlib: pyplot tutorial (построение графиков). [25]
- scikit-learn: load_iris, train_test_split, StandardScaler. [16]
- TensorFlow: установка (совместимость Python) и Quickstart. [43]
- TensorFlow/Keras: Sequential и обучение через fit(). [11]
- Backprop (MSE-случай): учебные материалы с формулами δ и алгоритмом. [6]
- Русскоязычное объяснение backprop (учебник/справочник): например, раздел в Яндекс-учебнике по ML. [44]

Где полезно добавить изображения/схемы (рекомендация):

- Схема «слои и размерности тензоров» (в отчёт рядом с описанием X , $W1$, $b1$, ...).
 - Графики `loss_curve.png` и `acc_curve.png` как обязательные иллюстрации результата.
 - Mermaid-диаграммы из методички — как «быстрые» наглядные схемы алгоритма и структуры сети.
-

[1] [23] [34] Regression losses

https://keras.io/2/api/losses/regression_losses/?utm_source=chatgpt.com

[2] The Python Tutorial — Python 3.14.3 documentation

https://docs.python.org/3/tutorial/index.html?utm_source=chatgpt.com

[3] `numpy.matmul` — NumPy v2.5.dev0 Manual

https://numpy.org/devdocs/reference/generated/numpy.matmul.html?utm_source=chatgpt.com

[4] [16] `load_iris` — scikit-learn 1.7.2 documentation

https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_iris.html?utm_source=chatgpt.com

[5] fgosvo.ru

https://fgosvo.ru/uploadfiles/FGOS%20VO%203%2B%2B/Bak/090302_B_3_17102017.pdf

[6] [9] [36] Метод обратного распространения ошибки

<https://sok.susu.ru/courses/MachineLearnig/lectures/04%20Backpropagation.pdf>

[7] [18] [20] 12. Virtual Environments and Packages

https://docs.python.org/3/tutorial/venv.html?utm_source=chatgpt.com

[8] [11] [26] [27] The Sequential model | TensorFlow Core

https://www.tensorflow.org/guide/keras/sequential_model?utm_source=chatgpt.com

[10] [15] [25] [32] Pyplot tutorial — Matplotlib 3.10.8 documentation

https://matplotlib.org/stable/tutorials/pyplot.html?utm_source=chatgpt.com

[12] `StandardScaler` — scikit-learn 1.7.2 documentation

https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html?utm_source=chatgpt.com

[13] [31] `train_test_split` — scikit-learn 1.7.2 documentation

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html?utm_source=chatgpt.com

[14] [17] [43] Install TensorFlow with pip

https://www.tensorflow.org/install/pip?utm_source=chatgpt.com

[19] [30] Install packages in a virtual environment using pip and venv

https://packaging.python.org/guides/installing-using-pip-and-virtual-environments/?utm_source=chatgpt.com

[21] Managing Python — conda 26.1.1 documentation

https://docs.conda.io/projects/conda/en/stable/user-guide/tasks/manage-python.html?utm_source=chatgpt.com

[22] [37] Installing pip packages - Anaconda

https://www.anaconda.com/docs/getting-started/working-with-conda/packages/pip-install?utm_source=chatgpt.com

[24] Random Generator — NumPy v2.1 Manual

https://numpy.org/doc/2.1/reference/random/generator.html?utm_source=chatgpt.com

[28] Losses

https://keras.io/api/losses/?utm_source=chatgpt.com

[29] Training & evaluation with the built-in methods

https://www.tensorflow.org/guide/keras/training_with_built_in_methods?utm_source=chatgpt.com

[33] tf.keras.Sequential | TensorFlow v2.16.1

https://www.tensorflow.org/api_docs/python/tf/keras/Sequential?utm_source=chatgpt.com

[35] venv — Creation of virtual environments

https://docs.python.org/3/library/venv.html?utm_source=chatgpt.com

[38] [40] Art. 5 GDPR – Principles relating to processing of personal ...

https://gdpr-info.eu/art-5-gdpr/?utm_source=chatgpt.com

[39] Федеральный закон от 27 июля 2006 г. N 152-ФЗ

https://fstec.ru/dokumenty/vse-dokumenty/zakony/federalnyj-zakon-ot-27-iyulya-2006-g-n-152-fz?utm_source=chatgpt.com

[41] AI principles

https://www.oecd.org/en/topics/sub-issues/ai-principles.html?utm_source=chatgpt.com

[42] Broadcasting — NumPy v2.4 Manual

https://numpy.org/doc/stable/user/basics.broadcasting.html?utm_source=chatgpt.com

[44] 5.3 Метод обратного распространения ошибки

https://education.yandex.ru/handbook/ml/article/metod-obratnogo-rasprostraneniya-oshibki?utm_source=chatgpt.com